

Stabilne skojarzenia

Uniwersalny algorytm Gale'a-Shapleya dla grafów dwudzielnych

Jakub Adamczyk

7 czerwca 2026

1 Opis problemu i warianty topologiczne

Problem stabilnego skojarzenia (ang. *Stable Matching Problem*) polega na znalezieniu optymalnego przypisania jeden-do-jednego między dwoma rozłącznymi zbiorami (np. zbiór kandydatów A i zbiór stanowisk B). Każdy element posiada uporządkowaną listę preferencji. Celem jest znalezienie **stabilnego skojarzenia**, czyli przypisania, w którym nie istnieje żadna para, która wolałaby połączyć się ze sobą z pominięciem swoich aktualnych partnerów.

Algorytm dostosowano do obsługi trzech głównych wariantów problemu:

1. **Równe zbiory, graf pełny ($K_{N,N}$):** Klasyczny wariant. Każdy uczestnik posiada pełną listę preferencji i znajduje przypisanie.
2. **Nierówne zbiory ($K_{N,M}$):** Problem Szpitali i Rezydentów ($N \neq M$). Część wierzchołków z konieczności pozostaje nieprzypisana.
3. **Graf niepełny (SMI):** Wierzchołki posiadają niepełne listy preferencji (wolą pozostać wolne, niż zaakceptować połączenie z kimś niechcianym).

2 Analiza algorytmu Gale'a-Shapleya

- **Działanie:** Zbiór wolnych elementów z A oświadcza się w dół swoich list preferencji. Zbiór B przyjmuje oferty, ale wymienia partnerów, gdy otrzyma lepszą propozycję. Element z A , który wyczerpie swoją listę, trwale pozostaje bez przypisania.
- **Złożoność:** Dzięki odwróconej macierzy preferencji zbioru B (słownik w języku Python), weryfikacja partnerów odbywa się w czasie stałym $\mathcal{O}(1)$.
- **Poprawność:** Algorytm wymusza monotoniczność decyzji. Zbiór B porzuca partnerów wyłącznie na rzecz lepszych ofert, co eliminuje ryzyko niestabilności w układzie końcowym.

3 Szczegółowa analiza krok po kroku (Ślad algorytmu)

Aby w pełni zrozumieć mechanikę wymiany partnerów, prześledźmy przebieg algorytmu dla klasycznego przypadku z równymi zbiorami (Test 1 z sekcji z implementacją). Zbiory A i B liczą po 3 elementy.

Listy preferencji (od najlepszego wyboru):

- A1: B1, B2, B3 | B1: A2, A3, A1
- A2: B1, B3, B2 | B2: A1, A2, A3
- A3: B2, B1, B3 | B3: A1, A2, A3

Przebieg symulacji (Krok po kroku):

1. **A1** jest wolny. Składa propozycję pierwszej osobie ze swojej listy: **B1**. Z racji, że B1 jest wolne, powstaje tymczasowa para (A1, B1).
2. **A2** jest wolny. Składa propozycję **B1**. B1 jest w parze z A1, ale sprawdza swój ranking. B1 woli A2 od A1, więc zrywa z A1 i łączy się z A2. Powstaje para (A2, B1). A1 wraca do puli wolnych.
3. **A3** jest wolny. Składa propozycję **B2**. B2 jest wolne. Powstaje para (A3, B2).
4. **A1** (ponownie wolny) idzie do kolejnej osoby z listy: **B2**. B2 jest w parze z A3, ale woli A1. B2 zrywa z A3 i łączy się z A1. Powstaje para (A1, B2). A3 wraca do puli wolnych.
5. **A3** (ponownie wolny) składa propozycję swojemu drugiemu wyborowi: **B1**. B1 jest w parze z A2 i woli obecnego partnera. B1 odrzuca ofertę A3. A3 pozostaje wolny.
6. **A3** (nadal wolny) składa propozycję **B3**. B3 jest wolne. Powstaje para (A3, B3).

Kolejka wolnych elementów ze zbioru A jest pusta. Algorytm kończy działanie, zwracając w pełni stabilne skojarzenie: (A2, B1), (A1, B2), (A3, B3).

4 Implementacja w języku Python z przykładami

```
1 def gale_shapley(preferencje_A, preferencje_B):
2     wolni_A = list(preferencje_A.keys())
3     nastepna_prop = {a: 0 for a in preferencje_A}
4     parowanie_B = {b: None for b in preferencje_B}
5
6     # Odwrocona macierz rankingow dla predkosci O(1)
7     rank_B = {}
8     for b, lista in preferencje_B.items():
9         rank_B[b] = {a: i for i, a in enumerate(lista)}
10
11     while wolni_A:
12         a = wolni_A[0]
13         i_kand = nastepna_prop[a]
14
15         # Ochrona: brak krawedzi lub nierowne zbiory
16         if i_kand >= len(preferencje_A[a]):
17             wolni_A.pop(0)
18             continue
19
20         b = preferencje_A[a][i_kand]
21         nastepna_prop[a] += 1
22         stary_partner = parowanie_B.get(b)
23
24         if stary_partner is None:
25             parowanie_B[b] = a
26             wolni_A.pop(0)
27         else:
28             # Uzycie float('inf') dla brakujacych kandydatow
29             poz_nowy = rank_B[b].get(a, float('inf'))
30             poz_stary = rank_B[b].get(stary_partner, float('inf'))
31
32             if poz_nowy < poz_stary:
33                 parowanie_B[b] = a
34                 wolni_A.pop(0)
35                 wolni_A.append(stary_partner)
36
37     return {a: b for b, a in parowanie_B.items() if a is not None}
38
39 # TEST 1: Rowne zbiory (Pelny graf)
40 pA1 = {'A1': ['B1', 'B2', 'B3'], 'A2': ['B1', 'B3', 'B2'], 'A3': ['B2',
41     'B1', 'B3']}
42 pB1 = {'B1': ['A2', 'A3', 'A1'], 'B2': ['A1', 'A2', 'A3'], 'B3': ['A1',
43     'A2', 'A3']}
44 w1 = gale_shapley(pA1, pB1)
45
46 # TEST 2: Nierowne zbiory (Brakuje miejsc w B)
47 pA2 = {'A1': ['B1', 'B2'], 'A2': ['B1', 'B2'], 'A3': ['B2', 'B1']}
48 pB2 = {'B1': ['A1', 'A2', 'A3'], 'B2': ['A2', 'A1', 'A3']}
49 w2 = gale_shapley(pA2, pB2)
50
51 # TEST 3: Graf niepelny (A1 ma tylko 1 preferencje)
52 pA3 = {'A1': ['B1'], 'A2': ['B1', 'B2'], 'A3': ['B2', 'B3']}
53 pB3 = {'B1': ['A2', 'A1'], 'B2': ['A3', 'A2'], 'B3': ['A3']}
54 w3 = gale_shapley(pA3, pB3)
```

Listing 1: Uniwersalny Algorytm Gale'a-Shapleya oraz 3 testy

5 Wyniki działania programu (Konsola)

Poniżej zaprezentowano zrzut ekranu z konsoli, przedstawiający rzeczywiste wyniki działania zaimplementowanych testów. Potwierdzają one uniwersalność algorytmu w starciu z pełnymi grafami, nierównymi zbiorami oraz brakującymi krawędziami.

```
=====
PRZYPADEK 1: Równe zbiory, Pełny graf (3 na 3)
Oczekiwany wynik: Wszyscy znajdują parę.
=====
A1 <-> B2
A2 <-> B1
A3 <-> B3

=====
PRZYPADEK 2: Nierówne zbiory (3 kandydatów, 2 miejsca)
Oczekiwany wynik: Brak miejsc, jeden kandydat (A3) zostaje singlem.
=====
A1 <-> B1
A2 <-> B2
Pozostali singlem: A3

=====
PRZYPADEK 3: Graf niepełny (3 na 3, brakujące krawędzie)
Sytuacja: A1 zgadza się tylko na B1. B1 woli A2. A1 odpada.
Oczekiwany wynik: Równe zbiory, a mimo to A1 i B3 zostają sami.
=====
A2 <-> B1
A3 <-> B2
Zostali bez przypisania:
- Kandydaci: A1
- Stanowiska: B3
```

Rysunek 1: Zrzut ekranu z wynikami działania algorytmu dla trzech wariantów.